Factory

Building the intuition

Every design pattern solves a specific recurring problem. Before you memorize what a Factory Method looks like, you need to feel *why* it exists — the pain it relieves. Let's start from zero.

THE PROBLEM — YOUR FIRST INSTINCT

Imagine you're building a notification system. Your backend needs to send an Email notification. Simple enough:

```
public class NotificationService {
    public void send(String message) {
        EmailNotification notif = new EmailNotification();  // tightly coupled
        notif.deliver(message);
    }
}
```

This works. You ship it. Three months later, product says: *"We need SMS too."* You change the code. Six months later: *"Push notifications."* Another change. A year later: *"Slack and WhatsApp."*

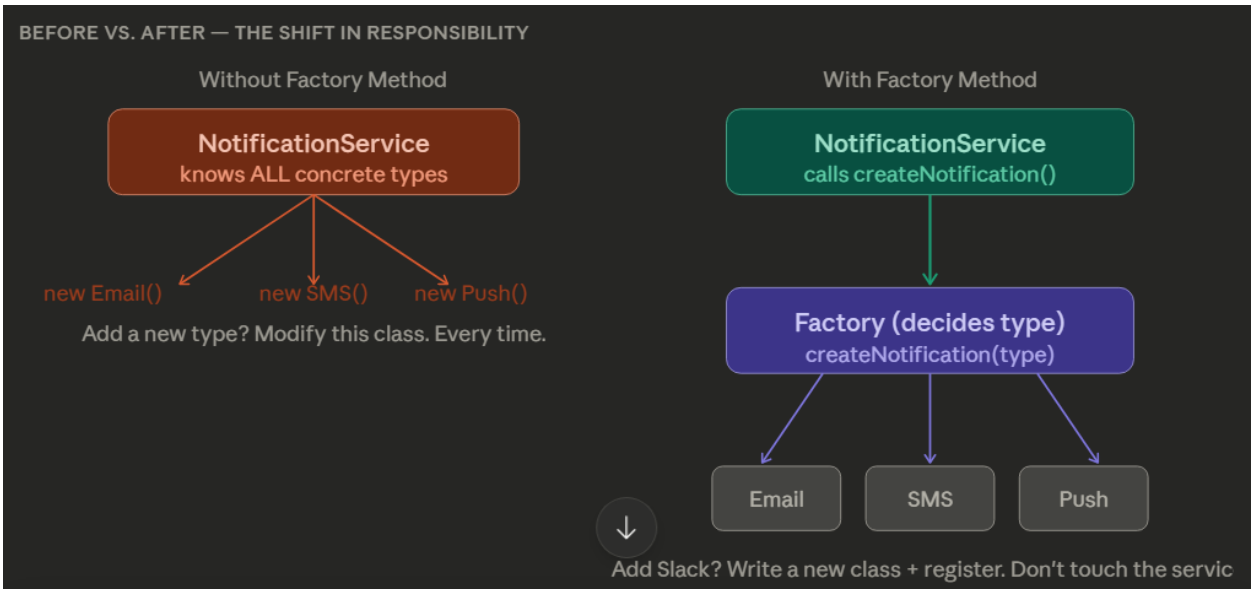
> Every time a new notification type appears, you reopen and modify `NotificationService`. You are violating the **Open/Closed Principle** — classes should be open for extension but closed for modification. Your service knows too much about which concrete class to create.

THE REAL PROBLEM — TIGHT COUPLING TO `NEW`

The root issue is that `new EmailNotification()` is a hard dependency. It bakes a specific concrete type into your business logic. When types multiply or change, your logic must change too. The word `new` is a dependency nail you hammer into your code.

THE ANALOGY — A LOGISTICS COMPANY

> Think of a logistics company that ships parcels. It doesn't care whether it uses a truck, a ship, or a plane — it just needs *a transport* that can *deliver*. The logistics coordinator asks the dispatch centre: *"Get me transport for this route."* The dispatch centre figures out which vehicle to create based on the destination, weight, and urgency. The coordinator never calls `new Truck()` directly.
>
> The dispatch centre is your **Factory Method**. The coordinator is your business logic. The transport types (truck, ship, plane) are your concrete products.



> The key insight: Factory Method moves the `new` keyword — and the decision of *which* class to instantiate — out of your business logic and into a dedicated place. Your business code works against an *interface*, not against concrete classes. That single shift makes your system extensible, testable, and maintainable.

The four components

Every Factory Method implementation — in any language, any domain — consists of exactly four roles. Understand these roles, and you can recognize and implement the pattern anywhere.

interface

Product

The common interface (or abstract class) that all created objects implement. Defines what the created objects can *do*, not what they *are*. Your business logic knows only this interface — it never touches the concrete types directly.

Example: `Notification` interface with a `send()` method.

class

Concrete Product

The actual implementations of the Product interface. Each one knows how to do the job in its specific way. You can have as many as you need — adding a new one never changes existing code.

Example: `EmailNotification` , `SmsNotification` , `PushNotification` .

abstract

Creator

Declares the factory method — usually abstract. Contains the business logic that works with the Product interface. It does not know which Concrete Product will be created. This separation is the whole point of the pattern.

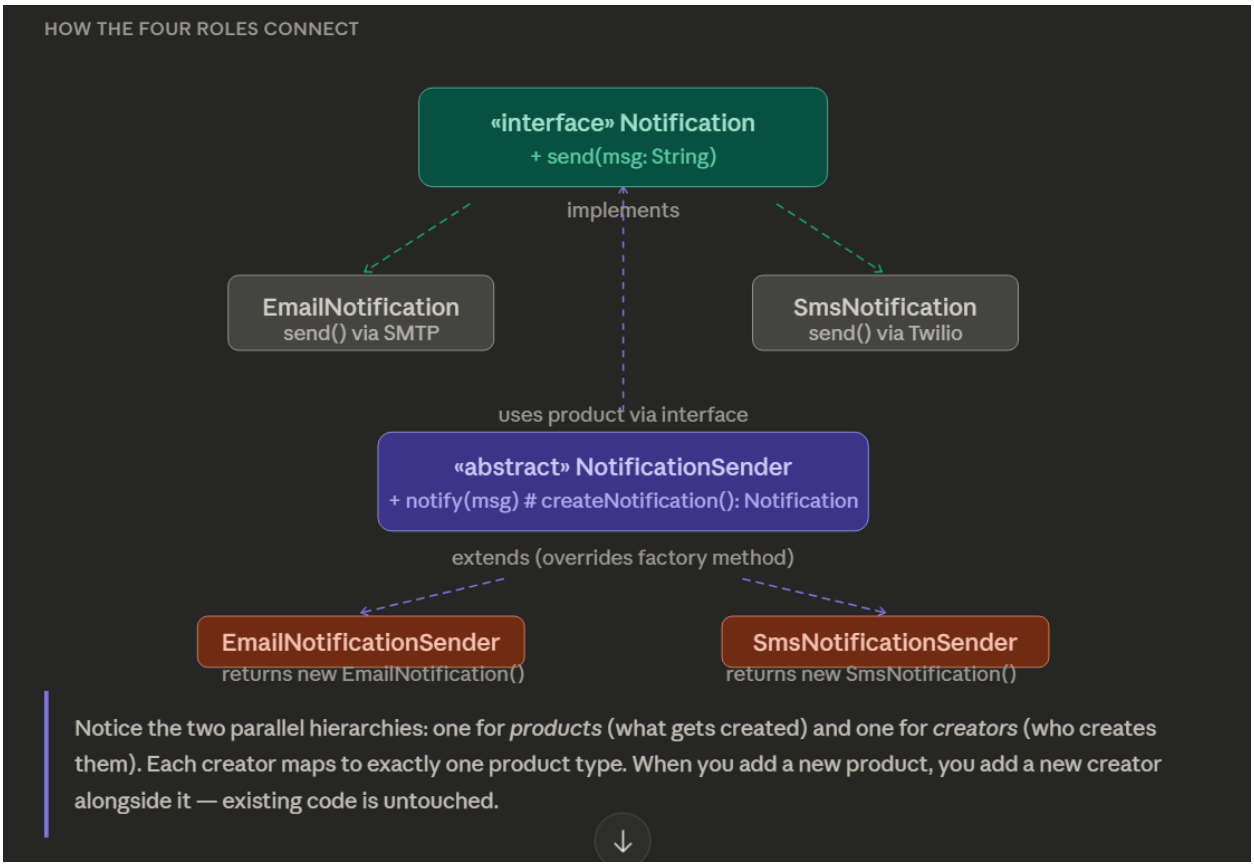
Example: `NotificationSender` abstract class with abstract `createNotification()` .

class

Concrete Creator

Overrides the factory method and returns an instance of a specific Concrete Product. This is the only place where `new ConcreteProduct()` is called. Each concrete creator maps to one product type.

Example: `EmailNotificationSender` returns `new EmailNotification()` .



Notice the two parallel hierarchies: one for *products* (what gets created) and one for *creators* (who creates them). Each creator maps to exactly one product type. When you add a new product, you add a new creator alongside it — existing code is untouched.

Implementation flow

1. Classic (Subclass)

The textbook version. One creator subclass per product. Demonstrates the pattern most clearly — use this to learn the structure.

```
// — Step 1: Product interface ————————————————————————
public interface Notification {
    void send(String message);
}
```

```java
// ── Step 2: Concrete Products ──────────────────────────────
public class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Sending EMAIL: " + message);
        // SMTP logic here
    }
}

public class SmsNotification implements Notification {
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
        // Twilio API call here
    }
}
```

```java
// ── Step 3: Abstract Creator ───────────────────────────────
// Business logic lives here. Never touches concrete types.
public abstract class NotificationSender {

    // THE factory method — subclasses must implement this
    protected abstract Notification createNotification();

    // Business logic uses only the Product interface
    public void notify(String message) {
        Notification notif = createNotification();  // delegate creation
        notif.send(message);
        // could also log, retry, audit here — all against the interface
    }
}
```

```java
// ── Step 4: Concrete Creators ──────────────────────────────
public class EmailNotificationSender extends NotificationSender {
    protected Notification createNotification() {
        return new EmailNotification();  // only concrete class instantiation
    }
}

public class SmsNotificationSender extends NotificationSender {
    protected Notification createNotification() {
        return new SmsNotification();
    }
}
```

```java
// ── Usage ──────────────────────────────────────────────────
NotificationSender sender = new EmailNotificationSender();
sender.notify("Your order has shipped!");
// Swap to SMS: just change which sender you instantiate. Logic unchanged.
```

THE MENTAL MODEL — STEP BY STEP FLOW

2. Parametrized (Registry)

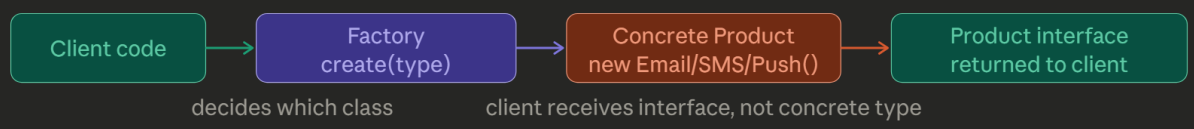
> More practical for real backends. A single factory class takes a type parameter and uses a registry (map) to look up the right creator. Adding a new type = one new class + one map entry. Zero changes to existing classes.

```java
// ── Product interface (same as before) ──────────────────
public interface Notification {
    void send(String message);
}

// ── Concrete Products (same as before) ──────────────────
// EmailNotification, SmsNotification, PushNotification...
```

```java
// ── Parametrized Factory ──────────────────────────────
public class NotificationFactory {

    // Registry: maps a type string → supplier (factory function)
    private static final Map<String, Supplier<Notification>> REGISTRY
        = new HashMap<>();

    // Register default types at class-load time
    static {
        REGISTRY.put("email", EmailNotification::new);
        REGISTRY.put("sms",   SmsNotification::new);
        REGISTRY.put("push",  PushNotification::new);
    }

    // Allow new types to be registered at runtime (plugin point)
    public static void register(String type, Supplier<Notification> supplier) {
        REGISTRY.put(type, supplier);
    }

    // THE factory method
    public static Notification create(String type) {
        Supplier<Notification> supplier = REGISTRY.get(type.toLowerCase());
        if (supplier == null) {
            throw new IllegalArgumentException("Unknown notification type: " + type);
        }
        return supplier.get();
    }
}
```

```java
// ── Usage ──────────────────────────────────────────────────────
String channel = user.getPreferredChannel();   // "email" | "sms" | "push"
Notification notif = NotificationFactory.create(channel);
notif.send("Payment confirmed.");

// Adding Slack later — existing classes untouched:
NotificationFactory.register("slack", SlackNotification::new);
```

THE MENTAL MODEL — STEP BY STEP FLOW



Use cases in backend development

These are places you will genuinely encounter Factory Method in production backend systems. The common thread: multiple implementations of the same behavior, chosen at runtime based on context.

- **Notification / messaging channels**
  Email, SMS, Push, WhatsApp, Slack — all implement a `Notification` interface. The factory selects the channel based on user preference or event type. New channels are added as new classes without touching business logic. This is the canonical example because it maps 1:1 to the pattern structure.

- **Payment gateway integration**
  Stripe, PayPal, Razorpay, Braintree all implement a `PaymentGateway` interface with `charge()`, `refund()`, `verify()`. The factory selects based on the user's region, currency, or plan. Switching or adding a gateway is isolated to one new class — the checkout logic never changes.

- **Authentication strategy selection**
  JWT, OAuth2, API key, SAML, LDAP all implement an `AuthStrategy` interface. The factory reads the client type (mobile app vs. enterprise SSO vs. third-party API) from the request and returns the right authenticator. Your security filter never sees concrete auth classes.

- **File / document parsers**
  A document ingestion service needs to parse CSV, JSON, XML, XLSX, and PDF files. Each implements a `DocumentParser` interface. The factory inspects the MIME type or file extension and returns the right parser. Business logic just calls `parser.parse(file)` — the format is irrelevant to it.

- **Report generation (format-based)**
  A reporting service exports to PDF, Excel, or HTML. Each implements a `ReportExporter` interface. The factory reads the `format` query param and creates the right exporter. Downstream business logic calls `exporter.export(data)` — same code, different output format.

- **Storage provider abstraction**
  S3, Google Cloud Storage, Azure Blob Storage, local filesystem — all implement a `StorageProvider` interface. The factory reads config to decide which provider to create. Your service just calls `storage.upload(file)`. Switching cloud providers requires zero changes to application logic.

- **Database dialect / ORM adapters**
  Hibernate uses a dialect factory internally — `PostgreSQLDialect`, `MySQLDialect`, `OracleDialect` all extend a base `Dialect`. The dialect factory reads your `spring.datasource.url` and creates the right one. You write HQL/JPQL once; the dialect factory handles SQL generation differences per database.

Pros & cons

Factory Method is one of the most useful patterns you'll learn — but like any tool, it can be misapplied. Here's how a senior engineer thinks about it.

Pros

- Open/Closed Principle — add new products without changing any existing code
- Single Responsibility — object creation logic lives in one dedicated place
- Loose coupling — callers depend on interfaces, never on concrete classes
- Testability — inject a mock product via the factory in unit tests
- Runtime flexibility — choose the product type based on config, user data, or env
- Scales gracefully — adding 10 new types costs no more than adding 1

Cons

- More classes — every new product type adds two classes (product + creator)
- Indirection — harder for new developers to trace which class actually runs
- Overkill for small systems — if you have 2 types that never change, it's over-engineering
- Concrete creators are tightly coupled to their concrete product
- Parametrized variant risks type-string typos unless you use an enum as the key

THE OVER-ENGINEERING TRAP — A REAL CONVERSATION

JUNIOR MINDSET

"I'll use Factory Method everywhere — it's a design pattern, so it must be better."

SENIOR MINDSET

"Is there more than one implementation today, or a clear roadmap for more? If not, just use `new`. I can always refactor to Factory Method when the second type arrives — that's the right trigger."

ANTI-PATTERN

Building a factory for a payment service that only ever uses Stripe, and has no plans to change. Extra classes, extra indirection, no benefit.

GOOD JUDGMENT

Building a notification factory from day one, because even if you only have email today, experience tells you SMS and push will come within 3 months. Factory Method is a smart bet here.

THE ONE-LINE DECISION RULE

> If you have a `switch` or `if/else if` block that decides which class to instantiate based on a type, and that block will grow over time — that is your signal to introduce a Factory Method. The pattern replaces the branching with a registry or subclass dispatch.

FACTORY METHOD VS. RELATED PATTERNS

Factory Method vs. Abstract Factory

Factory Method creates one product. Abstract Factory creates *families* of related products. Think: Factory Method = one notification channel; Abstract Factory = a full theme kit (button + icon + color + font).

Factory Method vs. Builder

Factory Method answers *"which type?"* — it selects among different implementations of an interface. Builder answers *"how is it configured?"* — it assembles a single complex object step by step.

Factory Method vs. Strategy

Very close siblings. Strategy swaps algorithms at runtime on an existing object. Factory Method creates new objects of the right type upfront. Strategy is behavioral; Factory Method is creational.